



ARTIFICIAL INTELLIGENCE

Class XI

STUDENT HANDBOOK



Subject Code:843

UNIT 3: Python Programming

Title: Python Programming	Approach: Group Discussion, Hands on Practice using the software
Summary: This unit will introduce students to the fundamentals/ basics of Python programming language, its history, evolution, operators, variables, constants, lists, strings, iterative and select statements. Students will explore three essential Python libraries: NumPy, Pandas, and Scikit-learn. Students will learn how Python is used to create programs. They will also learn how to use NumPy for numerical computing, Pandas for data manipulation and analysis, and Scikit-learn for implementing machine learning algorithms.	
Learning Objectives: Students will be able to 1. Understand the basics of python programming language- tokens, data types, lists, string manipulation, iterative and decision statements. 2. Learn how to use NumPy for mathematical operations and numerical computing. 3. Explore Pandas for data manipulation, analysis, and exploration of structured data. 4. Gain proficiency in using Scikit-learn for implementing machine learning algorithms, including classification. 5. Develop the skills necessary to use Python libraries effectively in Data Science and machine learning projects.	
Key concepts: 1. Basics of python programming language 2. Understanding of character sets, tokens, modes, operators and data types 3. Control Statements 4. CSV Files 5. Libraries – NumPy, Pandas, Scikit-learn	
Learning Outcomes: Students will be able to – 1. Explain the basics of python programming language and write programs with basic concepts of tokens. 2. Use selective and iterative statements effectively. 3. Gain practical knowledge on how to use the libraries efficiently.	
Pre-requisites: Reasonable fluency in English language and basic computer skills	



Introduction to Python

Python is a general-purpose, high level programming language. It was created by Guido van Rossum, and released in 1991. Python got its name from a BBC comedy series – “Monty Python’s Flying Circus”



Features of Python

- High Level language
- Interpreted Language
- Free and Open Source
- Platform Independent (Cross-Platform) – runs virtually in every platform if a compatible python interpreter is installed.
- Easy to use and learn – simple syntax similar to human language.
- Variety of Python Editors – Python IDLE, PyCharm, Anaconda, Spyder
- Python can process all characters of ASCII and UNICODE.
- Widely used in many different domains and industries.



Python Editors

There are various editors and Integrated Development Environments (IDEs) that you can use to work with Python. Some popular options are PyCharm, Spyder, Jupyter Notebook, IDLE etc. Let us look how we can work with Jupyter Notebook.

Jupyter Notebook is an open-source web application that allows you to create and share documents containing live code, equations, visualizations, and narrative text. It's widely used in data science and research. It can be installed using Anaconda or with pip. For more details of installation use the link

<https://docs.jupyter.org/en/latest/install/notebook-classic.html>

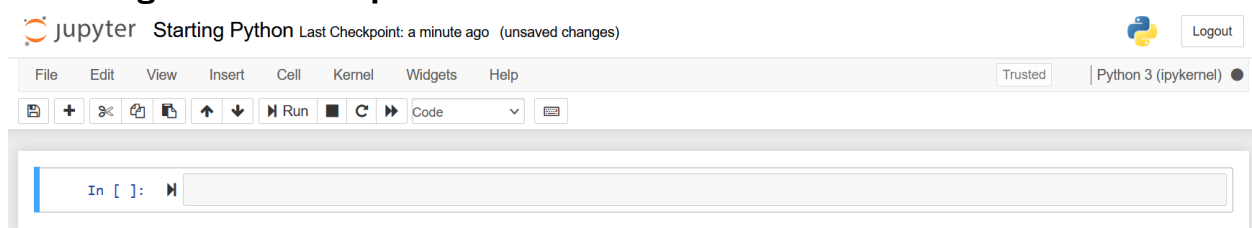
Those who are familiar with Python, open the command prompt in administrative mode and type

```
pip install notebook
```

To run the notebook, Open the command prompt and type

```
jupyter notebook
```

Following window will open



You can type the code in the cell provided. Then click  to see the output just below it.

```
In [1]: ▶ print ( " Hai all this is how to display a text")
```

Hai all this is how to display a text

```
In [4]: ▶ #To Add two numbers
num1=10
num2=20
print (num1 + num2)
```

30

Output – We can work on multiple programs in same window



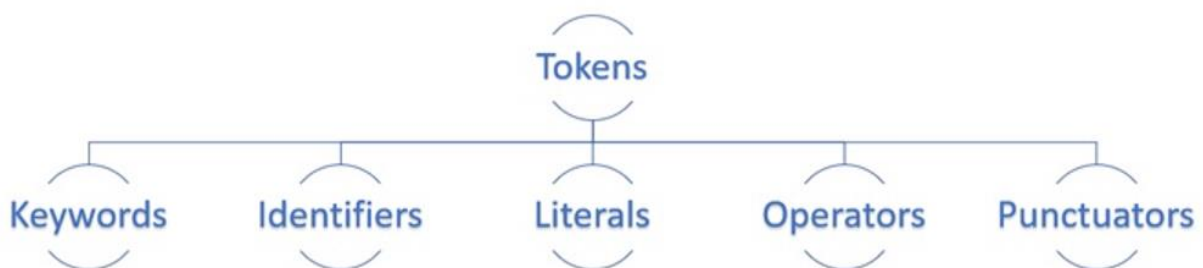
Point to be Noted

Another way to install and start using Jupyter Notebook is through Anaconda. It is the most widely used Python distribution for data science and comes pre-loaded with all the most popular libraries and tools. Jupyter notebook can easily be accessed using the Anaconda Prompt with the help of a local host.



Getting Started with Python Programs

Python program consists of Tokens. It is the smallest unit of a program that the interpreter or compiler recognizes. Tokens consist of keywords, identifiers, literals, operators, and punctuators. They serve as the building blocks of Python code, forming the syntactic structure that the interpreter parses and executes. During lexical analysis, the Python interpreter breaks down the source code into tokens, facilitating subsequent parsing and interpretation processes.



<https://www.studytrigger.com/wp-content/uploads/2022/08/Tokens-in-Python.jpg>

Keywords

Reserved words used for special purpose. List of keywords are given below.

False	None	True	for	in	or	while
and	class	elif	from	is	pass	with
as	continue	else	global	lambda	raise	yield
assert	def	except	if	nonlocal	return	async
break	del	finally	import	not	try	await

Identifier

An identifier is a name used to identify a variable, function, class, module or other object. Generally, keywords (list given above) are not used as variables. Identifiers cannot start with digit and also it can't contain any special characters except underscore.

Literals:

Literals are the raw data values that are explicitly specified in a program. Different types of Literals in Python are String Literal, Numeric Literal (Numbers), Boolean Literal (True & False), Special Literal (None) and Literal Collections.

Operators:

Operators are symbols or keywords that perform operations on operands to produce a result. Python supports a wide range of operators:

- | | |
|---|--|
| <ul style="list-style-type: none">• Arithmetic operators (+, -, *, /, %)• Relational operators (==, !=, <, >, <=, >=)• Assignment operators (=, +=, -=) | <ul style="list-style-type: none">• Logical operators (and, or, not)• Bitwise operators (&, , ^, <<, >>)• Identity operators (is, is not)• Membership operators (in, not in) |
|---|--|

Punctuators:

Common punctuators in Python include

: () [] { } , ; . ` ' ' ' " " / \ & @ ! ? | ~ etc.

Example

In [2]:	<pre>#Finding Square root of a number with the function sqrt() of math Library import math num = 625 root = math.sqrt(num) print("Square root= ", root)</pre>
output	Square root= 25.0

Tokens in the above program are given below

- Keyword - import
- Identifier - num , root (Here it can be said as variables also)
- Literal - 625
- Operator - =
- Punctuator - " " , () .

Point to be Noted

In the above program

-  `print ()` is used to display the output on the screen
-  `#` symbol is used to write comments which are used to increase readability and will not be executed
-  `import` statement is used to load the functions from the library (math)
-  Variables – Named labels whose value can be used and processed during the execution of the program.

Sample Program-1

Display the string “National Animal-Tiger” on the screen

```
In [1]: ▶ print("National animal - Tiger")
```

output

National animal - Tiger

Sample Program-2

Write a program to calculate the area of a rectangle given the length and breadth are 50 and 20 respectively.

```
In [1]: ▶ length = 12  
        breadth = 7  
        area = length * breadth  
        print("Area of Rectangle=", area)
```

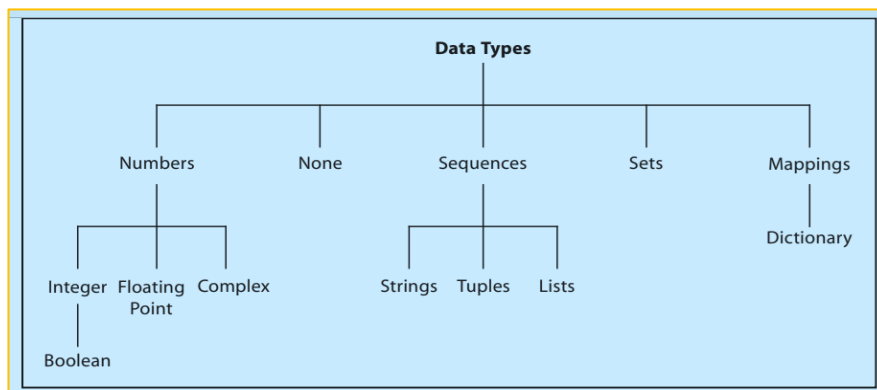
output

Area of Rectangle= 84

Data Types:

Data types are the classification or categorization of data items. It represents the kind of value that tells what operations can be performed on a particular data. Python supports Dynamic Typing. A variable pointing to a value of certain data type can be made to point to a value/object of another data type. This is called Dynamic Typing.

The following are the standard or built-in data types in Python:



Data Type Description

Integer	Stores whole number	a=10
Boolean	Boolean is used to represent the truth values of the expressions. It has two values True & False	Result = True
Floating point	Stores numbers with fractional part	x=5.5
Complex	Stores a number having real and imaginary part	num=a+bj
String	Immutable sequences (After creation values cannot be changed in-place) Stores text enclosed in single or double quotes	name= "Ria")
List	Mutable sequences (After creation values can be changed in-place) Stores list of comma separated values of any data type between square []	lst=[25, 15.6, "car", "XY"]
Tuple	Immutable sequence (After creation values cannot be changed in-place) Stores list of comma separated values of any data type between parentheses ()	tup=(11, 12.3, "abc")
Set	Set is an unordered collection of values, of any type, with no duplicate entry.	s = { 25, 3, 3.5}
Dictionary	Unordered set of comma-separated key:value pairs within braces { }	dict= { 1 : "One", 2: "Two", 3: "Three"}



Accepting values from the user

The input() function retrieves text from the user by prompting them with a string argument. For instance:

```
name = input("What is your name?")
```

Return type of input function is string. So, to receive values of other types we have to use conversion functions together with input function.

Sample Program-3

Write a program to read name and marks of a student and display the total mark.

```
In [3]: ▶ name=input("Enter Student's Name")
        m1=float(input("Enter the Mark of English"))
        m2=float(input("Enter the Mark of Artificial Intelligence"))
        m3=float(input("Enter the Mark of Maths"))
        Total=m1+m2+m3
        print("Name : ", name)
        print("Total Marks : ", Total)
```

output

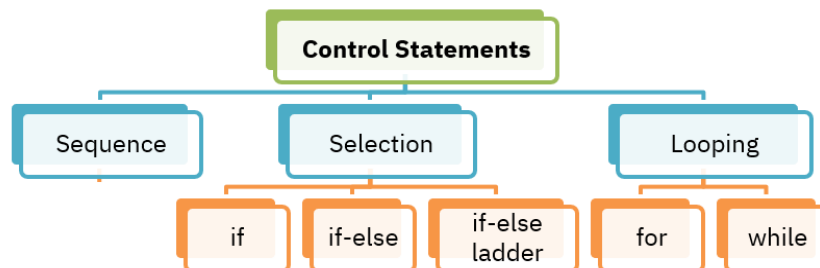
```
Enter Student's Name  M J Anakha
Enter the Mark of English  99
Enter the Mark of Artificial Intelligence  100
Enter the Mark of Maths  96
Name :    M J Anakha
Total Marks :  295.0
```

In the above example float() is used to convert the datatype into floating point. The explicit conversion of an operand to a specific type is called **type casting**.



Control flow statements in Python

Till now, the programs you've created have followed a basic, step-by-step progression, where each statement executes in sequence, every time. However, there are many practical programs where we have to selectively execute specific sections of the code or iterate over parts of the program. This capability is achieved through selective statements and looping statements.



Selection Statement

The if/ if..else statement evaluates test expression and the statements written below will execute if the condition is true otherwise the statements below else will get executed. Indentation is used to separate the blocks.

Syntax:

```
if <test expression>:  
    Statements
```

```
if <test expression>:  
    Body of if  
else:  
    Body of else
```

```
if <test expression>:  
    Body of if  
elif <test expression>:  
    Body of elif  
else:  
    Body of else
```

Let's check out different examples to see the working of if and if-else statements

Sample Program-4

Asmita with her family went to a restaurant. Determine the choice of food according to the options she chooses from the main menu.

Case 1: All Members are vegetarians. They prefer to have veg food. No other options.
(menu-veg)

Program & Output

```
choice=input("Enter the choice of food")  
if choice == "veg":  
    print("Welcome to Vegetarian Food House")
```

```
Enter the choice of foodveg  
Welcome to Vegetarian Food House
```


Case 2: Family Members may choose non-vegetarian foods also if veg foods are not available. (menu-veg/Nonveg)

```
choice=input("Enter the choice of food")
if choice == "veg":
    print("Welcome to Vegetarian Food House")
else:
    print("Welcome to Non-vegetarian Foods")
```

```
Enter the choice of food Nonveg
Welcome to Non-vegetarian Foods
```

Case 3: Family members can choose from variety of options

```
print("Menu: Veg | Nonveg | Mixed")
choice=input("Enter the choice of food")
if choice == "veg":
    print("Welcome to Vegetarian Food House")
elif choice == "Nonveg":
    print("Welcome to Non-vegetarian Foods")
else:
    print("Welcome to your Favourite Choice of Foods")
```

```
Menu: Veg | Nonveg | Mixed
Enter the choice of food Mixed
Welcome to your Favourite Choice of Foods
```

Sample Program-5

Write a program to get the length of the sides of a triangle and determine whether it is equilateral triangle or isosceles triangle or scalene triangle,

```
# Sample program to classify a triangle based on its sides

# Get user input for the Lengths of the sides
side1 = float(input("Enter the length of side 1: "))
side2 = float(input("Enter the length of side 2: "))
side3 = float(input("Enter the length of side 3: "))

# Check the type of triangle based on the Lengths of its sides
if side1 == side2 == side3:
    print("It is an equilateral triangle.")
elif side1 == side2 or side1 == side3 or side2 == side3:
    print("It is an isosceles triangle.")
else:
    print("It is a scalene triangle.")
```

```
Enter the length of side 1: 7
Enter the length of side 2: 7
Enter the length of side 3: 11
It is an isosceles triangle.
```

Looping Statements

Looping statements in programming languages allow you to execute a block of code repeatedly. In Python, there are mainly two types of looping statements: for loop and while loop.

For loop

For loop iterates through a portion of a program based on a sequence, which is an ordered collection of items.

The **“for”** keyword is used to start the loop. The loop variable takes on each value in the specified sequence (e.g., list, string, range). The colon (:) at the end of the for statement indicates the start of the loop body. The statements within the loop body are executed for each iteration. Indentation is used to define the scope of the loop body. All statements indented under the for statement are considered part of the loop. It is advisable to utilize a for loop when the exact number of iterations is known in advance.

Syntax

```
for <control-variable> in <sequence/items in range>:  
    <statements inside body of the loop>
```

Example -1

```
for i in range(5):  
    print("Python")
```

```
Python  
Python  
Python  
Python  
Python
```

Example-2

```
for i in range(5):  
    print(i)
```

```
0  
1  
2  
3  
4
```

In the above program

- ✚ range (5) returns the values 0,1,2,3,4
- ✚ For each iteration of the loop variable i receives these values.
- ✚ First iteration of the loop i=0 (one time print(“Python”) executes, similarly with i=1,2,3,4 also print statement works.
- ✚ Whatever is given inside the loop executes repeatedly. In the first example, 5 times Python was printed, but in example-2, as i is to be printed it displayed 0 1 2 3 4

The **for** loop iterates over each item in the sequence until it reaches the end of the sequence or until the loop is terminated using a **break** statement. It's a powerful construct for iterating over collections of data and performing operations on each item.

Sample Program-6

Write a program to display even numbers and their squares between 100 and 110.

```
for num in range(100,110,2):  
    square=num*num  
    print(num, "squared is", square)
```

```
100 squared is 10000  
102 squared is 10404  
104 squared is 10816  
106 squared is 11236  
108 squared is 11664
```

Sample Program-7

Write a program to read a list, display each element and its type. (use type() to display the data type.)

```
lst = [ 25, "fruit", 17.7, ('a', 'b'), 100]  
for word in lst:  
    print (word, type(word))
```

```
25 <class 'int'>  
fruit <class 'str'>  
17.7 <class 'float'>  
( 'a', 'b' ) <class 'tuple'>  
100 <class 'int'>
```



Point to be Noted

In the above program

- ✚ the control variable word gets each element of the list. Hence in print statement each element and its type is displayed
- ✚ Same program can be written using the following code also
for i in range (len (lst)):
 print (lst[i] , type (lst[i])
Here we take i as index number, lst[0]= 25 & lst[-1] = 100

Sample Program-8

Write a program to read a string. Split the string into list of words and display each word.

```
Str="Iam studying in Jyothis Central School"  
wordlist=Str.split()  
print ("Words in list format", wordlist)  
for word in wordlist:  
    print(word)
```

```
Words in list format ['Iam', 'studying', 'in', 'Jyothis', 'Central', 'School']  
Iam  
studying  
in  
Jyothis  
Central  
School
```

Sample Program-9

Write a simple program to display the values stored in dictionary

```
dict = {'S1':"Bio-Math", 'S2':"Math-Comp", 'S3': "Bio-Psy", 'S4': "Math-AI"}  
for key in dict:  
    print(dict[key])
```

```
Bio-Math  
Math-Comp  
Bio-Psy  
Math-AI
```



UNDERSTANDING CSV file (Comma Separated Values)

CSV files are delimited files that store tabular data (data stored in rows and columns). It looks similar to spread sheets, but internally it is stored in a different format. In csv file, values are separated by comma. Data Sets used in AI programming are easily saved in csv format. Each line in a csv file is a data record. Each record consists of more than one fields(columns). The csv module of Python provides functionality to read and write tabular data in CSV format.

Let us see an example of opening, reading and writing formats for a file student.csv with file object file. student.csv contains the columns rollno, name and mark.

importing library	<code>import csv</code>
Opening in reading mode	<code>file= open("student.csv", "r")</code>
Opening in writing mode	<code>file= open("student.csv", "w")</code>
closing a file	<code>file.close()</code>
writing rows	<code>wr=csv.writer(file)</code> <code>wr.writerow([12, "Kalesh", 480])</code>
Reading rows	<code>details = csv.reader(file)</code> <code>for rec in details:</code> <code> print(rec)</code>

Sample Program-10

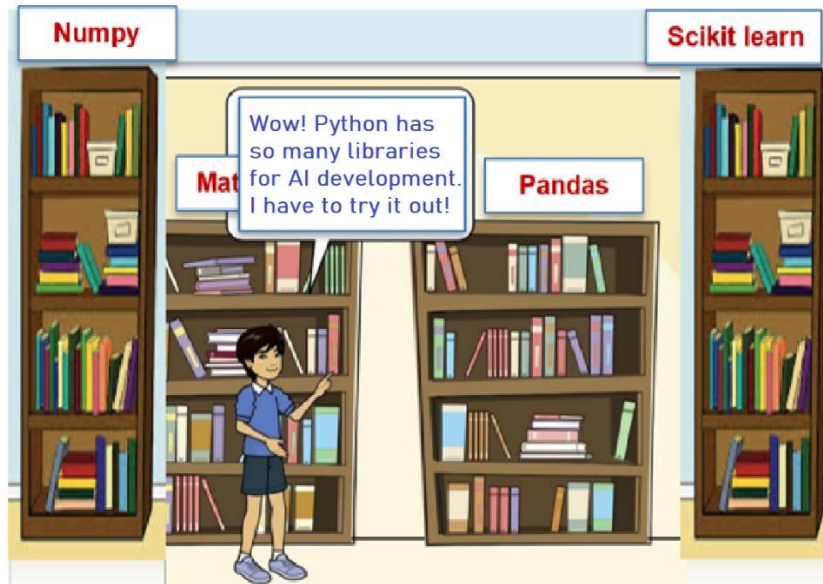
Write a Program to open a csv file students.csv and display its details

```
import csv  
file = open("D:\JPB\Python\students.csv", "r")  
details=csv.reader(file)  
for rec in details:  
    print(rec)
```

```
['RollNo', 'Name', 'class', 'TrName']  
['11', 'Akshith', 'II', 'Sruthy ']  
['12', 'Ashmitha', 'VII', 'Ruby']  
['13', 'M J Anakha', 'X', 'Jayasankar']
```

INTRODUCING LIBRARIES

A **library** in Python typically refers to a collection of reusable modules or functions that provide specific functionality. Libraries are designed to be used in various projects to simplify development by providing pre-written code for common tasks. Concept of libraries are very easy to understand.



In Python, functions are organized within libraries similar to how library books are arranged by subjects such as physics, computer science, and economics. For example, the "math" library contains numerous functions like `sqrt()`, `pow()`, `abs()`, and `sin()`, which facilitate mathematical operations and calculations. To utilize a library in a program, it must be imported. For example, if we wish to use the `sqrt()` function in our program, we include the statement "**import math**". This allows us to access and utilize the functionalities provided by the math library.

Python offers a vast array of libraries for various purposes, making it a versatile language for different domains such as web development, data analysis, machine learning, scientific computing, and more. Now, let us explore some libraries that are incredibly valuable in the realm of Artificial Intelligence.

NUMPY

NumPy, which stands for Numerical Python, is a powerful library in Python used for numerical computing. It is a general-purpose array-processing package. NumPy provides the **ndarray** (N-dimensional array) data structure, which represents arrays of any dimension. These arrays are homogeneous (all elements are of the same data type) and can contain elements of various numerical types (integers, floats, etc.)

Where and why do we use the NumPy library in Artificial Intelligence?

Suppose you have a dataset containing exam scores of students in various subjects, and you want to perform some basic analysis on this data. You can utilize NumPy arrays to store exam scores

for different subjects efficiently. With NumPy's array operations, you can perform various calculations such as calculating average scores for each subject, finding total scores for each student, calculating the overall average score across all subjects, identifying the highest and lowest scores. NumPy's array operations streamline these computations, making them both efficient and convenient. This makes NumPy an indispensable tool for data manipulation and analysis in data science applications.

NumPy can be installed using Python's package manager, pip.

```
pip install numpy
```

Creating a Numpy Array - Arrays in NumPy can be created by multiple ways. Some of the ways are programmed here:

➡ Using List of Tuples

```
import numpy as np
ar = np.array( [ (99, 88, 77), (44, 55, 66)] )
print ("Numpy Array:\n", ar)
```

```
Numpy Array:
[[99 88 77]
 [44 55 66]]
```

➡ Using values from the user (using empty())-- *The empty() function in Python is used*

```
import numpy as np
n=int(input("Enter the size of an array"))
ar=np.empty(n)
for i in range(n):
    ar[i]=int(input("Enter a number"))
print("Array\n", ar)
```

```
Enter the size of an array4
Enter a number34
Enter a number67
Enter a number85
Enter a number92
Array
[34. 67. 85. 92.]
```

to return a new array of a given size)

PANDAS

The name "Pandas" has a reference to both "Panel Data", and "Python Data Analysis". Pandas is a powerful and versatile library that simplifies tasks of data manipulation in Python . Pandas is built on top of the NumPy library which means that a lot of structures of NumPy are used or replicated in Pandas and Pandas is particularly well-suited for working with tabular data, such as spreadsheets or SQL tables. Its versatility and ease of use make it an essential tool for data analysts, scientists, and engineers working with structured data in Python.

Where and why do we use the Pandas library in Artificial Intelligence?

Suppose you have a dataset containing information about various marketing campaigns conducted by the company, such as campaign type, budget, duration, reach, engagement metrics, and sales performance. We use Pandas to load the dataset, display summary statistics, and perform group-wise analysis to understand the performance of different marketing campaigns. We then visualize the sales performance and average engagement metrics for each campaign type using Matplotlib, a popular plotting library in Python.

Pandas provides powerful data manipulation and aggregation functionalities, making it easy to perform complex analysis and generate insightful visualizations. This capability is invaluable in AI and data-driven decision-making processes, allowing businesses to gain actionable insights from their data.

Pandas can be installed using:

```
pip install pandas
```

Pandas generally provide two data structures for manipulating data, they are: Series and DataFrame.

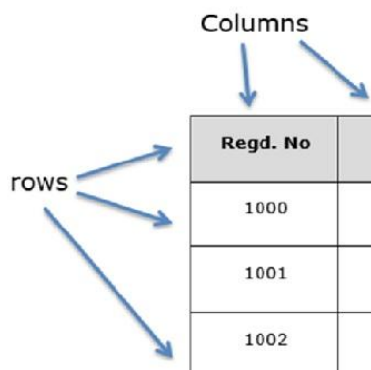
Series

A Series is a one-dimensional array containing a sequence of values of any data type (int, float, list, string, etc.) which by default have numeric data labels starting from zero. The data label associated with a particular value is called its **index**. We can also assign values of other data types as index. We can imagine a Pandas Series as a column in a spreadsheet as given here.

In data science, we often encounter datasets with two-dimensional structures. This is where Pandas DataFrames come into play.

A Data Frame is used when we need to work on multiple columns at a time, i.e., we need to process the tabular data.

Index	Data
0	Mark
1	Justin
2	John
3	Vicky



Regd. No	Name	Marks%
1000	Steve	86.29
1001	Mathew	91.63
1002	Jose	72.90
1003	Patty	69.23
1004	Vin	88.30

For example, the result of a class, items in a restaurant's menu, reservation chart of a train, etc.

A DataFrame is a two-dimensional labeled data structure like a table of MySQL. It contains rows and columns, and therefore has both a row and column index. Each column can have a different type of value such as numeric, string, boolean, etc., as in tables of a database.

Creation of DataFrame

There are several methods to create a DataFrame in Pandas, but here we will discuss two common approaches:

➡ Using NumPy ndarrays-

```
import numpy as np
import pandas as pd
array1 = np.array([10,20,30])
array2 = np.array([100,200,300])
array3 = np.array([-10,-20,-30])
dFrame = pd.DataFrame( [array1, array2, array3] , columns = ['col1', 'col2', 'col3'])
print(dFrame)
```

	col1	col2	col3
0	10	20	30
1	100	200	300
2	-10	-20	-30

➡ Using List of Dictionaries

```
listDict = [{'Dance':10, 'Music':20}, {'Dance':15,'Music':10,'Painting':20}, {'Painting': 12}]
a= pd.DataFrame(listDict, index=['X', 'XI', 'XII'])
print(a)
```

	Dance	Music	Painting
X	10.0	20.0	NaN
XI	15.0	10.0	20.0
XII	NaN	NaN	12.0

Point to be Noted

- ➔ Dictionary keys become column labels by default in a DataFrame, and the lists become the rows.
- ➔ NaN (Not a Number) is inserted if a corresponding value for a column is missing.
- ➔ Pandas uses isnull() function to identify NaN values in a DataFrame.

Dealing with Rows and Columns

Based on the DataFrame **'Result'** provided below, we can observe various operations related to rows and columns. Each operation statement is accompanied by its corresponding output from the Result DataFrame

DataFrame: **Result**

Result						
	Rajat	Amrita	Meenakshi	Rose	Karthika	
Maths	90	92	89	81	94	
Science	91	81	91	71	95	
Hindi	97	96	88	67	99	



Adding a New Column to a DataFrame:

We can add a new column 'Fathima', by mentioning column name as given below

```
Result['Fathima']=[89,78,76]
print(Result)
```

	Rajat	Amrita	Meenakshi	Rose	Karthika	Fathima
Maths	90	92	89	81	94	89
Science	91	81	91	71	95	78
Hindi	97	96	88	67	99	76



Adding a New Row to a DataFrame:

We can add a new row to a DataFrame using the DataFrame.loc[] method. Let us add marks for English subject in Result →

```
Result.loc['English'] = [90, 92, 89, 80, 90, 88]
print(Result)
```

	Rajat	Amrita	Meenakshi	Rose	Karthika	Fathima
Maths	90	92	89	81	94	89
Science	91	81	91	71	95	78
Hindi	97	96	88	67	99	76
English	90	92	89	80	90	88



Deleting Rows and Columns from a DataFrame:

We need to specify the names of the labels to be dropped and the axis from which they need to be dropped. To delete a row, the parameter axis is assigned the value 0 and for deleting a column, the parameter axis is assigned the value 1.

Deleting a row "Hindi"

```
Result = Result.drop('Hindi', axis=0)
print(Result)
```

	Rajat	Amrita	Meenakshi	Rose	Karthika	Fathima
Maths	90	92	89	81	94	89
Science	92	84	90	72	96	88
English	90	92	89	80	90	88

➡ **Delete the columns having labels 'Rajat', 'Meenakshi' and 'Karthika':**

```
Result = Result.drop(['Rajat', 'Meenakshi', 'Karthika'], axis=1)
print(Result)
```

	Amrita	Rose	Fathima
Maths	92	81	89
Science	84	72	88
English	92	80	88



Point to be Noted

During Data Analysis, DataFrame.drop() method is used to remove the rows and columns.



Accessing DataFrame Elements

Data elements in a DataFrame can be accessed using different ways. Two common ways of accessing are using loc and iloc. DataFrame.loc[] uses label names for accessing and DataFrame.iloc[] uses the index position for accessing the elements of a DataFrame. Let us check an example

```
Result.loc['Science']
```

```
Result.iloc[1]
```

```
Rajat      91
Amrita     81
Meenakshi  91
Rose       71
Karthika   95
Fathima    78
Name: Science, dtype: int64
```



Understanding Missing Values

Missing Data or Not Available data can occur when no information is provided for one or more items or for a whole unit. During Data Analysis, it is common for an object to have some missing attributes. If data is not collected properly it results in missing data.

In DataFrame it is stored as NaN (Not a Number). For example, while collecting data, some people may not fill all the fields while taking the survey. Sometimes, some attributes are not relevant to all.

Pandas provide a function isnull() to check whether any value is missing or not in the DataFrame. This function checks all attributes and returns True in case that attribute has missing values, otherwise returns False. Now, we can explore different operations related to missing values based on the DataFrame 'listDict' provided below.

	Dance	Music	Painting
X	10.0	20.0	NaN
XI	15.0	10.0	20.0
XII	NaN	NaN	12.0

StudCCA.isnull()



	Dance	Music	Painting
X	True	True	False
XI	True	True	True
XII	False	False	True

Point to be Noted

Finding any missing value in a column → `StudCCA['Music'].isnull().any() → True`
Finding total number of NaN → `StudCCA.isnull().sum() → 3`
Deleting entire row with NaN values → `StudCCA.dropna()`
Replacing NaN values (here by 1) → `StudCCA.fillna(1)`

Attributes of DataFrames

Attributes are the properties of a DataFrame that can be used to fetch data or any information related to a particular DataFrame.

The syntax of writing an attribute is:

DataFrame_name . attribute

Let us understand the attributes of DataFrames with the help of DataFrame Teacher
DataFrame:Teacher

	IX	X	XI	XII
Physics	Jayasankar	Shanthini	Sruthy	Anand Raj
Maths	Snitha	Haripriya	Praseetha	Sobhana Beegum
Artificial Intelligence	Noufiya	Rejila	Lekshmi	Jyoti

→ Displaying Row Indexes - Teacher.index

```
In [2]: Teacher.index
```

```
Out[2]: Index(['Physics', 'Maths', 'Artificial Intelligence'], dtype='object')
```

→ Displaying column Indexes - Teacher.columns

```
In [3]: Teacher.columns
```

```
Out[3]: Index(['IX', 'X', 'XI', 'XII'], dtype='object')
```

→ Displaying datatype of each - Teacher.dtypes

```
In [4]: Teacher.dtypes
```

```
Out[4]: IX      object
        X      object
        XI     object
        XII     object
        dtype: object
```

→ Displaying data in Numpy Array form - Teacher.values

```
In [5]: Teacher.values
```

```
Out[5]: array([[ 'Jayasankar', 'Shanthini', 'Sruthy', 'Anand Raj'],
               [ 'Snitha', 'Haripriya', 'Praseetha', 'Sobhana Beegum'],
               [ 'Noufiya', 'Rejila', 'Lekshmi', 'Jyoti']], dtype=object)
```



Displaying total number of rows and columns (row, column) - Teacher.shape

```
In [6]: Teacher.shape
```

```
Out[6]: (3, 4)
```



Displaying first n rows (here n = 2) - Teacher.head(2)

```
In [7]: Teacher.head(2)
```

```
Out[7]:
```

	IX	X	XI	XII
Physics	Jayasankar	Shanthini	Sruthy	Anand Raj
Maths	Snitha	Haripriya	Praseetha	Sobhana Beegum



Displaying last n rows (here n = 2) - Teacher.tail(2)

```
In [8]: Teacher.tail(2)
```

```
Out[8]:
```

	IX	X	XI	XII
Maths	Snitha	Haripriya	Praseetha	Sobhana Beegum
Artificial Intelligence	Noufiya	Rejila	Lekshmi	Jyoti

Importing and Exporting Data between CSV Files and DataFrames

We can create a DataFrame by importing data from CSV files. Similarly, we can also store or export data in a DataFrame as a .csv file.

Importing a CSV file to a DataFrame

Using the read_csv() function, you can import tabular data from CSV files into pandas dataframe by specifying a parameter value for the file name

Syntax: `pd.read_csv("filename.csv")`

Example: Reading file students.csv

```
import pandas as pd
import csv

df = pd.read_csv("D:\JPB\Python\students.csv", sep=";", header=0)
print(df)
```

	RollNo	Name	class	TrName
0	11	Akshith	II	Sruthy
1	12	Ashmitha	VII	Ruby
2	13	M J Anakha	X	Jayasankar



Point to be Noted

- read_csv() is used to read the csv file with its correct path.
- sep specifies whether the values are separated by comma, semicolon, tab, or any other character. The default value for sep is a space.
- The parameter header marks the start of the data to be fetched. header=0 implies that column names are inferred from the first line of the file. By default, header=0.

Exporting a DataFrame to a CSV file

We can use the `to_csv()` function to save a DataFrame to a text or csv file. For example, to save the DataFrame Teacher into csv file resultout, we should write

Teacher.to_csv(path_or_buf='C:/PANDAS/resultout.csv', sep=',')

When we open this file in any text editor or a spreadsheet, we will find the above data along with the row labels and the column headers, separated by comma.

Scikit-learn (Learn)

Note for Teachers: This topic can be taught after teaching the Machine Learning Unit.

Scikit-learn (Sklearn) is the most useful and robust library for machine learning in Python. It provides a selection of efficient tools for machine learning and statistical modeling via a consistent interface in Python. Sklearn is built on (relies heavily on) NumPy, SciPy and Matplotlib.

Key Features:

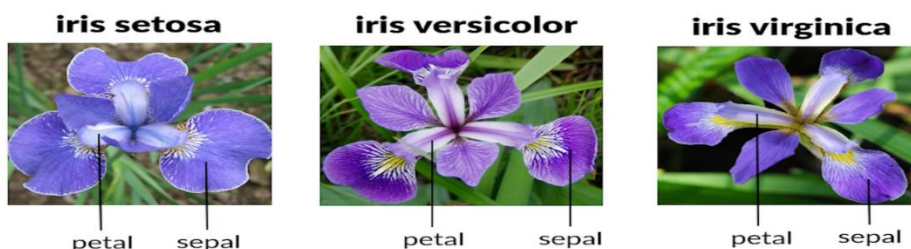
- Offers a wide range of supervised and unsupervised learning algorithms.
- Provides tools for model selection, evaluation, and validation.
- Supports various tasks such as classification, regression, clustering, dimensionality reduction, and more.
- Integrates seamlessly with other Python libraries like NumPy, SciPy, and Pandas.

Install scikit-learn using the statement

```
pip install scikit-learn
```

load_iris (In sklearn.datasets)

The Iris dataset is a classic and widely used dataset in machine learning, particularly for classification tasks. It comprises measurements of various characteristics of iris flowers, such as sepal length, sepal width, petal length, and petal width, along with the corresponding species of iris to which they belong. The dataset typically includes three species: setosa, versicolor, and virginica.



from sklearn.datasets import load_iris	importing iris dataset
iris = load_iris()	calls the “load_iris()” function to load the iris dataset
X = iris.data	X is a variable and assigned as feature vector. The feature vectors contain the input data for the machine learning model
y= iris.target	Y is a variable and assigned as target variable. The target variable contains the output or the variable we want to predict with the model.

Sample output – First 10 rows of X

```
Feature names: ['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']
Target names: ['setosa' 'versicolor' 'virginica']

First 10 rows of X:
[[5.1 3.5 1.4 0.2]
 [4.9 3.  1.4 0.2]
 [4.7 3.2 1.3 0.2]
 [4.6 3.1 1.5 0.2]
 [5.  3.6 1.4 0.2]
 [5.4 3.9 1.7 0.4]
 [4.6 3.4 1.4 0.3]
 [5.  3.4 1.5 0.2]
 [4.4 2.9 1.4 0.2]
 [4.9 3.1 1.5 0.1]]
```

Here, each row represents a sample (i.e., an iris flower), and each column represents a feature (i.e., a measurement of the flower).

For example, the first row [5.1 3.5 1.4 0.2] corresponds to an iris flower with the following measurements:

- Sepal length: 5.1 cm
- Sepal width: 3.5 cm
- Petal length: 1.4 cm
- Petal width: 0.2 cm

train_test_split (In sklearn.model_selection)

Datasets are usually split into training set and testing set. The training set is used to train the model and testing set is used to test the model.

Most common splitting ratio is 80: 20. (Training -80%, Testing-20%)

from sklearn.model_selection import train_test_split	importing train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, random_state = 1)	
X_train, y_train	the feature vectors and target variables of the training set respectively.
X_test, y_test	the feature vectors and target variables of the testing set respectively.
test_size = 0.2	specifies that 20% of the data will be used for testing, and the remaining 80% will be used for training.
random_state = 1	Ensures reproducibility by fixing the random seed. This means that every time you run the code, the same split will be generated.

KNeighborsClassifier (In sklearn.neighbors)

Scikit-learn has wide range of Machine Learning (ML) algorithms which have a consistent interface for fitting, predicting accuracy, recall etc. Here we are going to use KNN (K nearest neighbors) classifier.

from sklearn.neighbors import KNeighborsClassifier	importing KNeighborsClassifier (type of supervised learning algorithm used for classification tasks.)
knn = KNeighborsClassifier(n_neighbors =3)	we create an instance of the KNeighborsClassifier class . n_neighbors = 3 indicates that the classifier will consider the 3 nearest neighbors when making predictions. This is a hyperparameter that can be tuned to improve the performance of the classifier.
knn.fit(X_train, y_train)	trains the KNeighborsClassifier model using the fit method. it constructs a representation of the training data that allows it to make predictions based on the input features.
y_pred = knn.predict(X_test)	The knn object contains the trained model, make predictions on new, unseen data.

metrics

```
from sklearn import metrics
Accuracy = metrics.accuracy_score(y_test, y_pred))
```

This calculates the accuracy of the model by comparing the predicted target values (y_pred) with the actual target values (y_test). The accuracy_score represents the proportion of correctly predicted instances out of all instances in the testing set.

Scikit-learn offers a variety of modules that simplify the process of building, training, and evaluating machine learning models, making it a popular choice for various tasks in this domain. In our session, we utilized the 'load_iris()' function to load the Iris dataset. Upon loading, we split the dataset into training and test sets using the 'train_test_split' function. Subsequently, we trained our model using the K-Nearest Neighbors Classifier ('KNeighborsClassifier') and evaluated its performance using appropriate metrics. This workflow represents a typical data analysis pipeline in AI project development.

Now, to validate the model's predictive accuracy, we can use some sample data.

```
sample = [[5, 5, 3, 2], [2, 4, 3, 5]]
preds = knn.predict(sample)
for p in preds:
    pred_species.append(iris.target_names[p])
print("Predictions:", pred_species)
```

The provided code snippet demonstrates how to use the trained classifier to make predictions on sample data. After initializing the sample data as [5, 5, 3, 2], the classifier predicts the species of iris flowers based on these measurements. Finally, the predicted species are printed to the console.

This is a program that combines different parts of our project to make it complete and understandable.

```
from sklearn.datasets import load_iris
iris=load_iris()
X=iris.data
y=iris.target

from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=1)

from sklearn.neighbors import KNeighborsClassifier
knn = KNeighborsClassifier(n_neighbors =3)
knn.fit(X_train, y_train)
y_pred = knn.predict(X_test)

from sklearn import metrics
Accuracy = metrics.accuracy_score(y_test, y_pred)

sample = [[5, 5, 3, 2], [2, 4, 3, 5]]
preds = knn.predict(sample)
for p in preds:
    pred_species.append(iris.target_names[p])
print("Predictions:", pred_species)
```

Output-

```
Accuracy: 0.9833333333333333
Predictions: ['versicolor', 'virginica']
```

Using this model, we can identify the type of flower in the iris dataset. By analyzing the length and width of the sepals and petals, we can compare them with the features of the setosa, versicolor, and virginica species to determine the flower's species.



Point to be Noted

[Links to explore python more](#)

Tutorials

1. <https://www.programiz.com/python-programming>
2. <https://www.analyticsvidhya.com/blog/2021/05/data-types-in-python/>
3. <https://www.w3schools.com/python/default.asp>
4. <https://www.geeksforgeeks.org/pandas-tutorial/>
5. https://www.learnpython.org/en/Pandas_Basics
6. <https://www.geeksforgeeks.org/python-programming-language/>
7. <https://scikit-learn.org/stable/tutorial/basic/tutorial.html>
8. https://pandas.pydata.org/docs/user_guide/10min.html

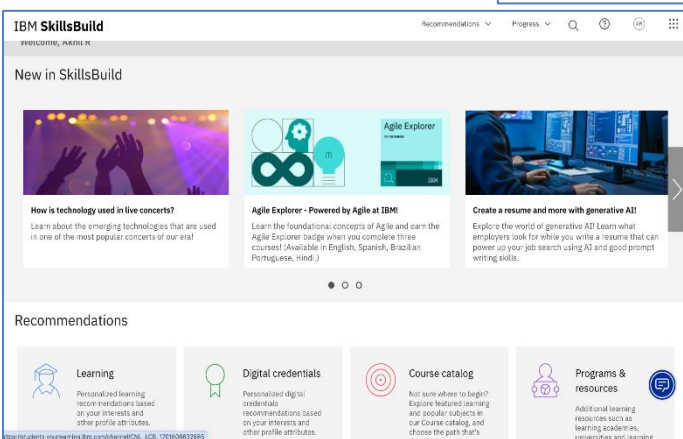
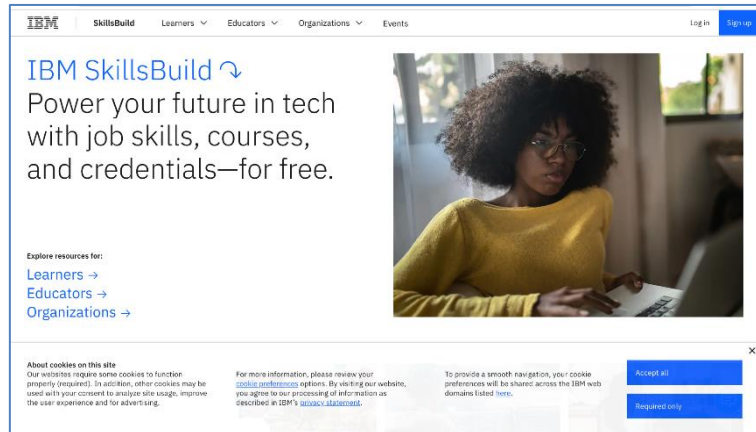
Courses

1. https://aistudent.community/single_course/2021
2. <https://www.kaggle.com/learn/pandas>
3. <https://www.udemy.com/course/pandas-with-python/>

Step-by-Step guide for students to use the IBM Skills Build website to learn Python:

Step 1: Visit the IBM SkillsBuild website using the link - <https://skillsbuild.org/> and sign up for an account.

Step 2: Locate and click on the "High School Student" option, then proceed to click on the "Sign Up" button.



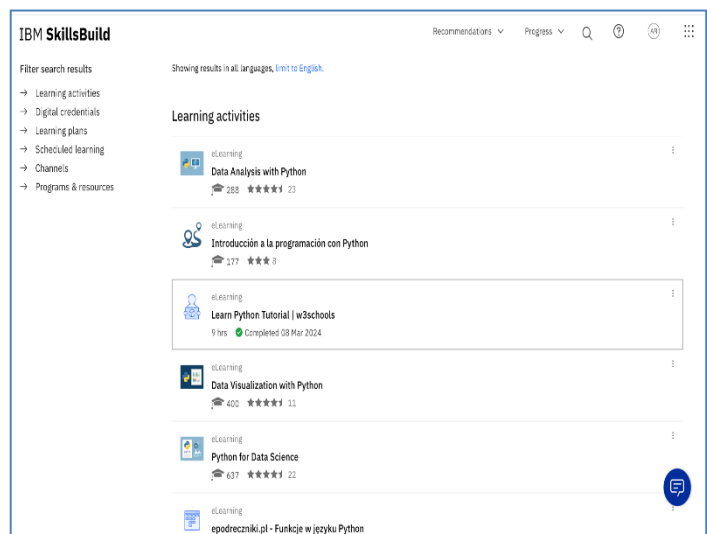
Step 3: Fill in the required information to create an account. You can sign up using your email address, LinkedIn ID, or IBM ID.

Step 4: Upon successfully completing this, you will be redirected to your dashboard. This is where you can explore a variety of courses.

Step 5: To start learning Python, use the search option at the top of the page and type in "Python" to find relevant courses.

Step 6: Browse, select a course, complete the tutorial and exercise.

Step 7: Monitor your progress on the IBM Skills Build platform and feel free to explore additional courses or resources to further enhance your understanding of Python and other related topics.



EXERCISES

A. Multiple choice questions

1. Identify the datatype `L = "45"`
a. String b. int c. float d. tuple
2. Which of the following function converts a string to an integer in python?
a. int(x) b. long(x) c. float(x) d. str(x)
3. Which special symbol is used to add comments in python?
a. \$ b. // c. /*... */ d. #
4. Which of the following variable is valid?
a. Str name b. 1str c. _str d. #Str
5. Elements in the list are enclosed in _____ brackets
a. () b. { } c. [] d. /* */
6. Index value of last element in list is _____
a. 0 b. -10 c. -1 d. 10
7. What will be the output of the following code/

```
a = [10,20,30,40,50]  
print([0])
```


a. 20 b. 50 c. 10 d. 40
8. Name the function that displays the data type of the variable.
a. data() b. type() c. datatype() d. int()
9. Which library helps in manipulating csv files?
a. files b. csv c. math d. print
10. Which keyword can be used to stop a loop?
a. stop b. break c. brake d. close
11. What is the primary data structure used in NumPy to represent arrays of any dimension?
a) Series b) DataFrame c) ndarray d) Panel
12. Which of the following is not a valid method to access elements of a Pandas DataFrame?
a) Using column names as attributes.
b) Using row and column labels with the .loc[] accessor.
c) Using integer-based indexing with the .iloc[] accessor.
d) Using the .get() method.
13. What is the purpose of the head() method in Pandas?
a) To display the first few rows of a DataFrame.
b) To display the last few rows of a DataFrame.
c) To count the number of rows in a DataFrame.
d) To perform aggregation operations on a DataFrame.
14. Which method is used to drop rows with missing values from a DataFrame in Pandas?
a) drop_rows() b) remove_missing() c) dropna() d) drop_missing_values

15. Which is not a module of Sklearn?

- a) load_iris b) train_test_split c) metrics d) Scikit

B. Answer the following questions

1. input() function accepts the value as string only. How can you convert string to int?
2. What are variables? What are the rules of declaring variables in Python?
3. What do you mean by type casting?
4. “Python supports dynamic typing”, True or False. Justify your answer.
5. Name any four features of python language.
6. Give examples for keywords.
7. Expand CSV.
8. How do you read data from a CSV file into a Pandas DataFrame?

C. Long Answer Questions

1. Describe the data types supported by Python, providing relevant examples.
2. Define an operator and provide examples of different operators along with their functions.

D. Practice Programs

1. Write a Tipper program where the user inputs the total restaurant bill. The program should then display two amounts: 15 percent tip and 20 percent tip.
2. Write a program to check whether the user is eligible for driving license or not.
3. Your father always gives his car for service after 15000 km. Check whether his car needs service or not. Read the kilometer reading from the user and give the output.
4. Write a program to display the first ten even natural numbers (use for loop).
5. Write a program to accept the Basic salary from the user and calculate the Net Salary.

$$\text{Net Salary} = \text{Basic Salary} + \text{HRA} + \text{DA} - \text{PF}$$

HRA=30% of Basic DA=20% of Basic PF=12% of Basic

6. Write a program to create series from an array in Python.
7. Consider the following admission.csv and answer the following questions:

Name	CLASS	Gender	Marks
Amit	10	M	75
Ashu	9	F	95
Abhinav	9	M	86
Ravi	10	M	57
Rashmi	11	F	78
Ramesh	10	M	72
Mohit	9	M	53
Manavi	10	F	47
Dhruv	9	M	76

- a. Create a dataframe from the admission.csv
- b. Display first 3 rows of the dataframe.
- c. Display the details of Ravi.
- d. Display the total number of rows and columns in the data frame.
- e. Display the column “Gender”.